

Terraform Associate Certification – Detailed Study Guide

This guide is designed to help learners prepare for the HashiCorp Certified Terraform Associate exam. It includes Terraform fundamentals, important commands, infrastructure as code concepts, hands-on labs, and practice questions. The goal is to give a practical understanding of Terraform along with the theory needed to pass the certification exam.

1. Infrastructure as Code (IaC)

Infrastructure as Code means managing and provisioning infrastructure through code instead of manual configuration. Terraform allows you to define infrastructure in configuration files that can be version controlled and reused.

Benefits:

- Automation
- Consistency
- Version control
- Reproducibility
- Faster deployments

2. Terraform Architecture

Terraform uses providers to interact with cloud platforms. A provider is responsible for understanding API interactions and exposing resources.

Core components:

- Terraform Core
- Providers (AWS, Azure, GCP)
- Resources
- State file

3. Terraform Workflow

Terraform follows a simple workflow:

1. Write configuration
2. Run terraform init
3. Run terraform plan
4. Run terraform apply
5. Manage lifecycle
6. Run terraform destroy when resources are no longer needed

4. Terraform Configuration Syntax

Terraform uses HashiCorp Configuration Language (HCL).

Example:

```
provider "aws" {  
  region = "ap-south-1"  
}
```

```
resource "aws_instance" "example" {  
  instance_type = "t2.micro"  
}
```

5. Variables and Outputs

Variables allow you to customize configurations.

Example variable:

```
variable "instance_type" {  
  default = "t2.micro"  
}
```

Outputs expose information after deployment.

6. Terraform State

Terraform state tracks the resources managed by Terraform. It maps configuration to real-world infrastructure. State files are critical because Terraform uses them to determine changes.

7. Remote Backends

Remote backends allow Terraform state to be stored remotely.

Example: AWS S3 backend

Benefits:

- Collaboration
- State locking
- Security

8. Terraform Modules

Modules allow reusable infrastructure code.

Example uses:

- VPC modules
- EC2 modules
- Networking modules

Modules help organize large infrastructure deployments.

9. Terraform Cloud

Terraform Cloud provides remote execution, state storage, and team collaboration features.

Capabilities:

- Remote runs
- State management
- Policy enforcement
- Workspaces

10. Workspaces

Workspaces allow multiple state environments within the same configuration.

Example:

- dev
- test
- prod

Hands-On Labs

- Lab 1: Install Terraform and verify using 'terraform version'.
- Lab 2: Create an AWS EC2 instance using Terraform.
- Lab 3: Create a VPC with Terraform.
- Lab 4: Configure an S3 backend for Terraform state.
- Lab 5: Build reusable Terraform modules.

Important Terraform CLI Commands

Command	Description
terraform init	Initialize working directory
terraform validate	Validate configuration
terraform plan	Preview infrastructure changes
terraform apply	Apply configuration
terraform destroy	Remove infrastructure

Practice Questions

- What command initializes a Terraform working directory?
- What is the purpose of the Terraform state file?
- What is a Terraform module?
- Which command previews changes before applying them?
- What backend is commonly used with AWS for remote state?
- What command deletes infrastructure managed by Terraform?
- What language does Terraform use for configuration?
- What is the purpose of variables in Terraform?
- What is Terraform Cloud used for?
- What is the purpose of workspaces?